

A.1 SHORTCOMINGS OF THE EXISTING MEMNNS

The Memory Networks models defined in (Weston et al., 2014) are one possible technique to try on our tasks, however there are several tasks which they are likely to fail on:

- They model sentences with a bag of words so are likely to fail on tasks such as the 2-argument (task 4) and 3-argument (task 5) relation problems.
- They perform only two max operations ($k = 2$) so they cannot handle questions involving more than two supporting facts such as tasks 3 and 7.
- Unless a RNN is employed in the R module, they are unable to provide multiple answers in the standard setting using eq. (3). This is required for the list (8) and path finding (19) tasks.

We therefore propose improvements to their model in the following section.

A.2 IMPROVING MEMORY NETWORKS

A.2.1 ADAPTIVE MEMORIES (AND RESPONSES)

We consider a variable number of supporting facts that is automatically adapted dependent on the question being asked. To do this we consider scoring a special fact $m_\emptyset$. Computation of supporting memories then becomes:

```

 $i = 1$ 
 $o_i = O(x, \mathbf{m})$ 
while  $o_i \neq m_\emptyset$  do
   $i \leftarrow i + 1$ 
   $o_i = O([x, m_{o_1}, \dots, m_{o_{i-1}}], \mathbf{m})$ 
end while

```

That is, we keep predicting supporting facts i , conditioning at each step on the previously found facts, until $m_\emptyset$ is predicted at which point we stop. $m_\emptyset$ has its own unique embedding vector, which is also learned. In practice we still impose a hard maximum number of loops in our experiments to avoid fail cases where the computation never stops (in our experiments we use a limit of 10).

Multiple Answers We use a similar trick for the response module as well in order to output multiple words. That is, we add a special word $w_\emptyset$ to the dictionary and predict word w_i on each iteration i conditional on the previous words, i.e., $w_i = R([x, m_{o_1}, \dots, m_{|o|}, w_i, \dots, w_{i-1}], w)$, until we predict $w_\emptyset$.

A.2.2 NONLINEAR SENTENCE MODELING

There are several ways of modeling sentences that go beyond a bag-of-words, and we explore three variants here. The simplest is a bag-of- N -grams, we consider $N = 1, 2$ and 3 in the bag. The main disadvantage of such a method is that the dictionary grows rapidly with N . We therefore consider an alternative neural network approach, which we call a **multilinear** map. Each word in a sentence is binned into one of P_{sz} positions with $p(i, l) = \lceil (iP_{sz})/l \rceil$ where i is the position of the word in a sentence of length l , and for each position we employ a $n \times n$ matrix $P_{p(i, l)}$. We then model the matching score with:

$$s(q, d) = E(q) \cdot E(d); \quad E(x) = \tanh\left(\sum_{i=1, \dots, l} P_{p(i, l)} \Phi_x(x_i)^\top U\right) \quad (5)$$

whereby we apply a linear map for each word dependent on its position, followed by a $\tanh$ non-linearity on the sum of mappings. Note that this is related to the model of (Yu et al., 2014) who consider tags rather than positions. While the results of this method are not shown in the main paper due to space restrictions, it performs similarly well to N -grams to and may be useful in real-world cases where N -grams cause the dictionary to be too large. Comparing to Table 3 MemNN with adaptive memories (AM) + multilinear obtains a mean performance of 93, the same as MemNNs with AM+NG+NL (i.e., using N -grams instead).

Finally, to assess the performance of nonlinear maps that do not model word position at all we also consider the following **nonlinear** embedding:

$$E(x) = \tanh(W \tanh(\Phi_x(x)^\top U)). \quad (6)$$

where W is a $n \times n$ matrix. This is similar to a classical two-layer neural network, but applied to both sides q and d of $s(q, d)$. We also consider the straight-forward combination of bag-of- N -grams followed by this nonlinearity.

B BASELINE USING EXTERNAL RESOURCES

We also built a classical cascade NLP system baseline using a structured SVM, which incorporates coreference resolution and semantic role labeling preprocessing steps, which are themselves trained on large amounts of costly labeled data. We first run the Stanford coreference system (Raghunathan et al., 2010) on the stories and each mention is then replaced with the first mention of its entity class. Second, the SENNA semantic role labeling system (SRL) (Collobert et al., 2011) is run, and we collect the set of arguments for each verb. We then define a ranking task for finding the supporting facts (trained using strong supervision):

$$o_1, o_2, o_3 = \arg \max_{o \in \mathcal{O}} S_O(x, f_{o_1}, f_{o_2}, f_{o_3}; \Theta)$$

where given the question x we find at most three supporting facts with indices o_i from the set of facts f in the story (we also consider selecting an “empty fact” for the case of less than three), and S_O is a linear scoring function with parameters Θ . Computing the argmax requires doing exhaustive search, unlike e.g. the MemNN method which is greedy. For scalability, we thus prune the set of possible matches by requiring that facts share one common non-determiner word with each other match or with x . S_O is constructed as a set of indicator features. For simplicity each of the features only looks at pairs of sentences, i.e. $S_O(x, f_{o_1}, f_{o_2}, f_{o_3}; \Theta) = \Theta * (g(x, f_{o_1}), g(x, f_{o_2}), g(x, f_{o_3}), g(f_{o_1}, f_{o_2}), g(f_{o_2}, f_{o_3}), g(f_{o_1}, f_{o_3}))$. The feature function g is made up of the following feature types, shown here for $g(f_{o_1}, f_{o_2})$: (1) Word pairs: One indicator variable for each pair of words in f_{o_1} and f_{o_2} . (2) Pair distance: Indicator for the distance between the sentence, i.e. $o_1 - o_2$. (3) Pair order: Indicator for the order of the sentence, i.e. $o_1 > o_2$. (4) SRL Verb Pair: Indicator variables for each pair of SRL verbs in f_{o_1} and f_{o_2} . (5) SRL Verb-Arg Pair: Indicator variables for each pair of SRL arguments in f_{o_1}, f_{o_2} and their corresponding verbs. After finding the supporting facts, we build a similar structured SVM for the response stage, also with features tuned for that goal: Words – indicator for each word in x , Word Pairs – indicator for each pair of words in x and supporting facts, and similar SRL Verb and SRL Verb-Arg Pair features as before.

Results are given in Table 3. The structured SVM, despite having access to external resources, does not perform better than MemNNs overall, still failing at 9 tasks. It does perform well on tasks 6, 9 and 10 where the hand-built feature conjunctions capture the necessary nonlinearities that the original MemNNs do not. However, it seems to do significantly worse on tasks requiring three (and sometimes, two) supporting facts (e.g. tasks 3, 16 and 2) presumably as ranking over so many possibilities introduces more mistakes. However, its non-greedy search does seem to help on other tasks, such as path finding (task 19) where search is very important.